

SQL Engine

[1. SQL Engine - Kompakte System-Referenz für KI-Modelle](#)

[2. Operationen \(DSL op\)](#)

[2.1. EXEC \(Kommando-Ausführung\)](#)

[2.1.1. Constraints: Dynamische Ergebnisfilterung](#)

[Struktur eines Constraints:](#)

[Beispiel: Anwendung von Constraints](#)

[Log-Output der Engine](#)

[2.1.2 Funktionsweise](#)

[Verbindungssteuerung & Konfiguration](#)

[2.2. ASSERT \(Validierung\)](#)

[Parameter-Verhalten:](#)

[2.3. BUFFER \(Datenextraktion\)](#)

[Parameter-Verhalten:](#)

[3. Referenz der SQL-Parameter](#)

[4.5. Praktische Anwendungsbeispiele & Best Practices](#)

[4.5.1 Adressierung: Zeilen und Spalten](#)

[4.5.2 Mengenprüfung \(Row Count\)](#)

[4.5.3 Unäre Validierungen \(Null & Empty\)](#)

[4.5.4 Vergleichsoperatoren](#)

[4.5.5 Intelligente Datumsprüfung](#)

[4.5.6 Mengenoperationen \(ANY_MATCH / ALL_MATCH\)](#)

[4.5.7 Integration: SQL & Buffer Engine](#)

Die SQL Engine ist eine Komponente des Test-Frameworks, die für die Interaktion mit relationalen Datenbanken konzipiert ist. Sie ermöglicht das Ausführen von Abfragen, die Validierung von Ergebnissen durch erweiterte Assertions sowie das Datenmanagement innerhalb von Testszenarien.

1. SQL Engine - Kompakte System-Referenz für KI-Modelle

```
1 # AGATE SQL-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 - type: SQL
5   op: EXEC
6   condition: "..."      # Optional: '{B[var]}' == 'val' (einfache Anf
7   command: |
8     SELECT ... FROM ...
9   response: "..."       # Name für den Ergebnis-Buffer
10  constraints:            # Optional: Filter-Logik (Pre-Filter)
11    - column: "..."
12      action: <EQUALS|...> # (Siehe ASSERT-Actions)
13      expected: "..."
14
15 - type: SQL
16   op: ASSERT
17   condition: "..."      # Optional: '{B[var]}' == 'val' (einfache Anf
18   response: "..."       # Referenz auf EXEC response
19   action: <EQUALS|NOT_EQUALS|GREATER_THAN|LESS_THAN|BETWEEN|CONTAINS|IS
20   column: "..."         # Spaltenname oder Index (0..N)
21   row: 0                 # Optional: Zeile (default: 0)
22   expected: "..."       # Erwarteter Wert (oder Range "X..Y")
```

```

23
24 - type: SQL
25   op: BUFFER
26   condition: "..."/>

```

2. Operationen (DSL op)

Operation	Beschreibung
EXEC	Führt ein SQL-Statement aus. SELECT -Ergebnisse werden im response -Buffer gespeichert. UPDATE / DELETE geben die Anzahl der betroffenen Zeilen zurück.
ASSERT	Validiert Daten aus dem response -Buffer basierend auf definierten Bedingungen (action).
BUFFER	Extrahiert den Wert einer spezifischen Zelle (row , column) und speichert ihn in einer Variable des Testfalls.

2.1. EXEC (Kommando-Ausführung)

Führt SQL-Befehl aus und speichert den gesamten Text-Output sowie den Exit-Code unter dem in response definierten Namen ab.

Parameter	Beschreibung	Erforderlich
command	Das auszuführende SQL-Statement (z.B. SELECT , UPDATE , DELETE).	Ja

response	Name (Buffer-Key), unter dem die Tabelle für spätere Schritte gespeichert wird.	Ja (nur für SELECT)
condition	Bedingte Ausführung (Schritt wird übersprungen, wenn <code>false</code>).	Nein
constraints: - column: "..." action: "..." expected: "..."	Eine Liste von Filterkriterien, die direkt nach der Abfrage auf das Result-Set angewendet werden (siehe Details unten).	Nein

Beispiel:

1	- type: SQL
2	op: EXEC
3	command: SELECT STATUS FROM SYSTEM_STATUS
4	response: res_demo

2.1.1. Constraints: Dynamische Ergebnisfilterung

Constraints fungieren als Pre-Filter für das Resultat einer EXEC-Operation. Bevor das Result-Set der Datenbank im `response` -Buffer abgelegt wird, filtert die Engine die Zeilen anhand der definierten Kriterien. Dies vereinfacht nachfolgende Validierungen (z.B. `ROW_COUNT` nach dem Filtern).

Struktur eines Constraints:

Ein Constraint besteht aus folgenden Feldern:

Unter-Parameter	Beschreibung
column	Name der Tabellenspalte (Case-Insensitive), auf die der Filter angewendet wird.
action	Die Filter-Operation (siehe unterstützte Filter-Aktionen).
expected	Der Vergleichswert (wird für <code>IS_NULL</code> und <code>IS_NOT_NULL</code>

nicht benötigt).

Beispiel: Anwendung von Constraints

Im folgenden Beispiel wird eine Liste von Status-Datensätzen abgerufen. Durch die **constraints** wird das Ergebnis automatisch so gefiltert, dass nur Zeilen übrig bleiben, die exakt den Kriterien entsprechen.

```
1 - type: SQL
2   op: EXEC
3   command: |
4     SELECT 'COMPLETED' as STATUS, 100 as AMOUNT FROM DUAL
5     UNION ALL
6     SELECT 'COMPLETED' as STATUS, 200 as AMOUNT FROM DUAL
7     UNION ALL
8     SELECT 'FAILED' as STATUS, 50 as AMOUNT FROM DUAL
9   response: aktivni_korisnici
10  constraints:
11    - column: "status"
12      action: "EQUALS"
13      expected: "COMPLETED"
14    - column: "amount"
15      action: "EQUALS"
16      expected: "100"
17
```

Log-Output der Engine

Die Engine dokumentiert den Filtervorgang transparent in den Logs:

```
1 >>> SQL EXEC:
2 >>>          SELECT 'COMPLETED' as STATUS, 100 as AMOUNT FROM DUAL
3 ...
4 >>> FILTERED: Constraints applied, rows remaining: 1
5 <<< +-----+
6 <<< | STATUS   | AMOUNT |
7 <<< +-----+
8 <<< | COMPLETED | 100    |
9 <<< +-----+
10 <<< [Total Rows: 1]
11 >>> Affected Rows: 1
12 >>> SQL STEP FINISHED | Status: SUCCESS | Duration: 6 ms
13
```

Wichtige Vorteile der Constraints:

- **Datenreduktion:** Sie reduzieren die Anzahl der im Speicher gehaltenen Zeilen.
- **Präzise Assertions:** Nachfolgende **ASSERT** -Schritte beziehen sich nur noch auf die gefilterten Daten, was die Testlogik vereinfacht.
- **Transparenz:** Da der Filtervorgang im Log explizit ausgegeben wird, ist sofort erkennbar, wie viele Datensätze nach dem Filtern für die weitere Verarbeitung zur Verfügung standen.

2.1.2 Funktionsweise

Verbindungssteuerung & Konfiguration

Der `DatabaseManager` agiert als Abstraktionsschicht zwischen dem Test-Framework und der physischen Datenbank. Die Verbindungsparameter werden nicht im Quellcode hinterlegt, sondern dynamisch zur Laufzeit über den `EnvironmentManager` geladen:

```
1 | env/env.config  
  
1 | DEMOS.database.user=D_DB  
2 | DEMOS.database.password=YouPassword  
3 | DEMOS.database.connectionString=jdbc:oracle:thin:@ora-host.db.co.at:1521
```

- **Aktuelle Unterstützung:** Der Framework-Kern ist aktuell **ausschließlich für Oracle-Datenbanken** optimiert und konfiguriert. Die JDBC-Treiber-Logik ist fest auf die Oracle-Infrastruktur ausgelegt.
- **Connectivity-Test (SQLPlus):** Zur Verifizierung, ob die Umgebung korrekt konfiguriert ist, sollte der Benutzer den Zugriff manuell via **SQLPlus** in der Konsole testen.

Beispiel für den Verbindungstest:

```
1 | sqlplus benutzername@service_name
```

Wenn der Login via SQLPlus erfolgreich ist, verfügt der Test-Client über die notwendigen Treiber und Berechtigungen für das Framework.

2.2. ASSERT (Validierung)

Prüft den Inhalt, die Struktur oder die Werte eines zuvor ausgeführten SQL-Statements (Result-Set). Schlägt eine Prüfung fehl, bricht der Test ab (Fail-Fast).

Parameter-Verhalten:

- **row (optional):** Gibt den Zeilenindex an (0 = erste Zeile). Wenn nicht definiert, wird standardmäßig **0** verwendet.
- **column:** Name oder Index der Spalte (erforderlich für die meisten Aktionen, außer `ROW_COUNT`).

Aktion	Beschreibung	Erforderliche Parameter
ROW_COUNT	Prüft die Gesamtanzahl der Zeilen im Result-Set. Spalte wird nicht benötigt.	expected
ALL_MATCH	Prüft, ob alle Werte in einer bestimmten	column, expected

	Spalte der Bedingung entsprechen.	
ANY_MATCH	Prüft, ob mindestens ein Wert in einer bestimmten Spalte der Bedingung entspricht.	column, expected
IS_NULL	Prüft, ob der Wert in der angegebenen Zelle <code>null</code> ist.	column
IS_NOT_NULL	Prüft, ob der Wert in der angegebenen Zelle existiert (nicht <code>null</code> ist).	column
IS_EMPTY	Prüft, ob die Zelle leer ist (Leerer String <code>""</code> oder <code>null</code>).	column
IS_NOT_EMPTY	Prüft, ob die Zelle Text enthält (nicht leer/null ist).	column
EQUALS	Prüft, ob der Zelleninhalt exakt sa vrednošću poklapa.	column, expected
NOT_EQUALS	Prüft, ob der Zelleninhalt vom erwarteten Wert abweicht.	column, expected
CONTAINS	Prüft, ob der erwartete Text im Zelleninhalt existiert.	column, expected
GREATER_THAN	Numerischer Vergleich: Prüft, ob der Zellwert größer als <code>expected</code> ist.	column, expected

LESS_THAN	Numerischer Vergleich: Prüft, ob der Zellwert kleiner als <code>expected</code> ist.	column, expected
GREATER_THAN_OR_EQUAL	Numerischer Vergleich: Prüft, ob der Zellwert größer oder gleich <code>expected</code> ist.	column, expected
LESS_THAN_OR_EQUAL	Numerischer Vergleich: Prüft, ob der Zellwert kleiner oder gleich <code>expected</code> ist.	column, expected
BETWEEN	Prüft, ob der numerische Wert in einem Bereich liegt (Format: <code>min..max</code> , z.B. <code>10..50</code>).	column, expected
DATE_EQUALS	Datum-Vergleich auf Tag-Ebene (Unterstützt <code>TODAY</code> , <code>YESTERDAY</code> oder konkretes Datum).	column, expected

Beispiele:

```

1 - response: my_sql_result
2   action: ROW_COUNT
3   expected: "5"

```

2.3. BUFFER (Datenextraktion)

Mit der BUFFER-Operation liest du gezielt einzelne Werte (Zellen) aus einem zuvor gespeicherten SQL-Result-Set (Tabelle) aus und speicherst diese in einer neuen Variablen (`name`).

Wichtige Regel: Verwende für `name` immer einen anderen Bezeichner als für `response`. Das Überschreiben eines originalen Response-Objekts (Tabelle) mit einem String führt zu Laufzeitfehlern.

Parameter-Verhalten:

- **row (optional):** Gibt den Zeilenindex an (0 = erste Zeile). Wenn nicht definiert, wird standardmäßig 0 verwendet.
- **column:** Name der Spalte oder der numerische Index der Spalte (0-basiert).
- **name:** Der Name der neuen Variable, in der das Ergebnis gespeichert wird.

(Hinweis: Das Feld `action` wird bei der SQL-BUFFER-Operation nicht verwendet.)

```
1 - type: SQL
2   op: BUFFER
3   row: 2
4   column: "1"
5   name: amount
6   response: res_multi2
```

3. Referenz der SQL-Parameter

Hier sind alle Parameter für die verschiedenen SQL-Schritte im Überblick:

Schritt	Parameter	Beschreibung
EXEC (Standard)	command	Das auszuführende SQL-Statement (z.B. <code>SELECT</code> , <code>UPDATE</code> , <code>INSERT</code>). Unterstützt dynamische Platzhalter.
	response	Name (Buffer-Key), unter dem das resultierende Result-Set (die Tabelle) im Kontext gespeichert wird.
	constraints	<i>Optional:</i> Liste von Bedingungen, um das

		Ergebnis-Set direkt nach der Abfrage zu filtern (z.B. <code>EQUALS</code> , <code>CONTAINS</code>).
	condition	<i>Optional:</i> Bedingung für die Ausführung. Der Schritt wird übersprungen (<code>SKIPPED</code>), wenn die Bedingung <code>false</code> ergibt.
BUFFER	response	Referenz auf das zuvor gespeicherte SQL-Result-Set (aus EXEC).
	column	Name der Spalte oder der numerische Index (0-basiert) der Spalte, aus der der Wert extrahiert werden soll.
	row	<i>Optional:</i> Zeilenindex (0 = erste Zeile). Wenn nicht angegeben, wird standardmäßig 0 verwendet.
	name	Name der neuen Puffer-Variable, unter der der extrahierte Einzelwert gespeichert wird.
	condition	<i>Optional:</i> Bedingung für die Ausführung. Der Schritt wird übersprungen (<code>SKIPPED</code>), wenn die

		Bedingung <code>false</code> ergibt.
ASSERT	response	Referenz auf das SQL-Result-Set (Tabelle) oder den Namen einer BUFFER-Variablen.
	action	Die Vergleichsoperation (z.B. <code>ROW_COUNT</code> , <code>EQUALS</code> , <code>IS_NULL</code> , <code>GREATER_THAN</code> , <code>BETWEEN</code> , <code>DATE_EQUALS</code> , <code>ALL_MATCH</code> , <code>ANY_MATCH</code>).
	column	Name oder Index der Spalte. Erforderlich für alle Aktionen außer <code>ROW_COUNT</code> .
	row	<i>Optional:</i> Zeilenindex für die Zielzelle (0 = erste Zeile). Wenn nicht angegeben, wird standardmäßig 0 verwendet.
	expected	Der erwartete Vergleichswert. Unterstützt Präfixe wie <code>NUMBER:</code> oder <code>DATE:SYSDATE</code> für dynamische Prüfungen.
	condition	<i>Optional:</i> Bedingung für die Ausführung. Der

Schritt wird
übersprungen
(`SKIPPED`), wenn die
Bedingung `false`
ergibt.

4.5. Praktische Anwendungsbeispiele & Best Practices

4.5.1 Adressierung: Zeilen und Spalten

Die Engine speichert Ergebnisse intern als Tabelle.

- **Indizierung:** Die Indizierung beginnt bei **0** (0..N).
- **Flexibilität:** Spalten können sowohl über ihren numerischen Index als auch über ihren Namen (Label aus dem SQL-Result) angesprochen werden. Der `row`-Parameter ist standardmäßig auf `0` gesetzt.

1 | YAML

```
1 # Beispiel: Zugriff auf Spalte via Index oder Name
2 - type: SQL
3   op: ASSERT
4   action: EQUALS
5   column: 4          # Zugriff via Index
6   expected: 5
7   response: res_demo
8
9 - type: SQL
10  op: ASSERT
11  action: EQUALS
12  row: 0             # Explizite Zeile (optional, da default 0)
13  column: "COUNTER" # Zugriff via Spaltenname
14  expected: "5"
15  response: res_demo
16
```

4.5.2 Mengenprüfung (Row Count)

Um die Anzahl der zurückgegebenen Datensätze zu validieren:

1 | YAML

```
1 - type: SQL
2   op: ASSERT
3   action: ROW_COUNT
4   expected: "1"
5   response: res_demo
6
```

4.5.3 Unäre Validierungen (Null & Empty)

Zur Prüfung des Zustands einzelner Zellen:

1 | YAML

```

1 # Prüfung auf NULL / NOT NULL
2 - type: SQL
3   op: ASSERT
4   action: IS_NOT_NULL
5   row: 0
6   column: 4
7   response: res_demo
8
9 # Prüfung auf Inhalt (Empty / Not Empty)
10 - type: SQL
11   op: ASSERT
12   action: IS_NOT_EMPTY
13   column: "COUNTER"
14   response: res_demo
15

```

4.5.4 Vergleichsoperatoren

Die Engine unterstützt klassische mathematische und logische Vergleiche:

```

1 | YAML
2
3 # Vergleichsoperatoren
4 - type: SQL
5   op: ASSERT
6   action: GREATER_THAN
7   column: "COUNTER"
8   expected: 3
9   response: res_demo
10
11 # Bereichsprüfung (Between)
12 - type: SQL
13   op: ASSERT
14   action: BETWEEN
15   column: "COUNTER"
16   expected: 3..5
17   response: res_demo
18

```

4.5.5 Intelligente Datumsprüfung

Um Abweichungen durch Zeitstempel zu vermeiden, bietet AGATE spezielle Keywords:

```

1 | YAML
2
3 # Prüfung mit Smart Date Keywords
4 - type: SQL
5   op: ASSERT
6   column: "DAY_COL"
7   action: DATE_EQUALS
8   expected: "TODAY" # oder "YESTERDAY"
9   response: res_demo3
10

```

4.5.6 Mengenoperationen (ANY_MATCH / ALL_MATCH)

Diese Operatoren prüfen, ob Bedingungen auf den gesamten Ergebnissatz zutreffen (ideal für Listen-Validierungen):

```

1 | YAML
2
3 # Prüft, ob mindestens ein Datensatz in der Spalte den Status 'FAILED' h

```

```

2 - type: SQL
3   op: ASSERT
4   column: "STATUS"
5   action: ANY_MATCH
6   expected: "FAILED"
7   response: res_multi
8

```

4.5.7 Integration: SQL & Buffer Engine

Ein mächtiges Feature ist die Verzahnung der SQL Engine mit der Buffer Engine. Ein Ergebnis einer SQL-Abfrage kann in eine Variable extrahiert und dort mit den Funktionen der Buffer Engine weiterverarbeitet werden.

```

1 | YAML
2
3 # 1. Wert aus SQL extrahieren
4 - type: SQL
5   op: BUFFER
6   row: 2
7   column: "1"
8   name: amount
9   response: res_multi2
10
11 # 2. Variable in der Buffer Engine prüfen
12 - type: BUFFER
13   op: ASSERT
14   name: "amount"
15   action: EQUALS
16   expected: "50"
17

```

Hinweis: Durch die konsequente Nutzung der `response`-Referenzen bleibt der Testfluss transparent und wartbar, da jeder Schritt explizit definiert, welche Datenmenge validiert werden soll.